

LLM Gateway

统一模型调用入口与出口，把模型调用规则集中到一个可治理的边界中

完成本节后，学员应当能够

- 1 说明 Agent 项目为什么需要独立的 LLM Gateway
- 2 使用 FastAPI 和 Pydantic 定义统一的模型调用入口与出口
- 3 使用 Provider Adapter 隔离 OpenAI Compatible 模型的地址、认证和调用细节
- 4 在 Gateway 中统一处理 Prompt 模板、Structured Output 和 Streaming
- 5 区分可重试错误与不可重试错误，实现有限重试和备用模型 fallback
- 6 记录模型、Prompt 版本、Token、Cost、Latency 和调用状态

前面已经学会调用模型，为什么还要增加 LLM Gateway?

LLM Gateway 的价值不是多转发一次 HTTP，而是把模型调用规则集中到一个可治理的边界中。



为什么 Gateway 要先定义自己的请求协议？

如果直接传供应商原始请求

如果上层直接传供应商的原始请求，Gateway 只是一个透明代理，模型差异仍然会泄漏到 Agent 中。

Gateway 定义独立请求协议

Gateway 拥有自己的 Pydantic 请求模型，上层 Agent 只提交平台模型名、模板选择、消息和业务 Schema——供应商差异被隔离在 Gateway 内部。

核心原则：协议隔离 — 上层不感知供应商，Gateway 不暴露实现

为什么 OpenAI Compatible 接口仍然需要 Provider Adapter?

兼容协议减少了适配成本，但 **Base URL**、**模型名**、**API Key** 和 **Structured Output** 能力仍然需要统一管理。

URL

Base URL

不同供应商地址不同，需要集中配置

ID

模型名

平台名到供应商名的映射管理

KEY

API Key

密钥不出 Gateway，上层无感知

JSON

Structured Output

能力差异需适配 json_schema / json_object

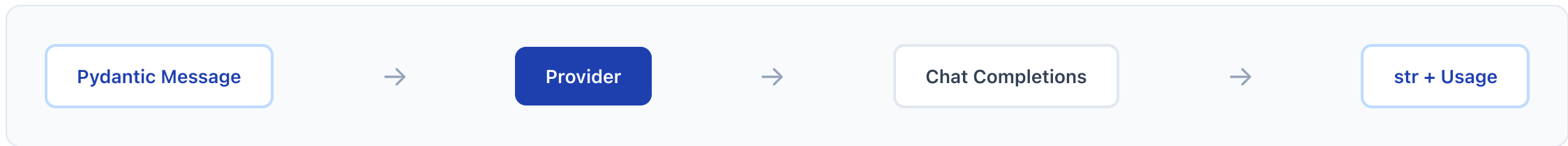
怎样把统一请求转换成真实模型调用？

1 Provider 转换与调用

Provider 把 Pydantic Message 转为普通字典，再调用 OpenAI Compatible Chat Completions。响应回来后，只提取完整文本与输入、输出 Token。

2 SDK 对象不离开 Provider

SDK 的 completion 对象不会离开 Provider。Gateway 的其他部分只消费 str 和 Usage，因此不会和供应商响应类型耦合。



为什么 Streaming 必须边接收边转发？

真正的流式代理应该收到一个增量块，就立即向下游发送一个增量块。



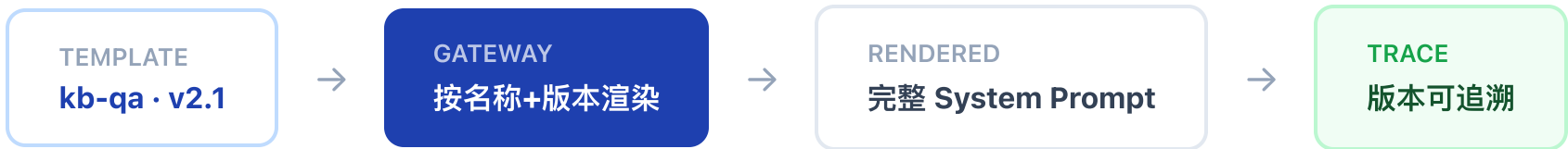
为什么 Prompt 模板要由 Gateway 管理？

如果由各 Agent 自己管理

如果每个业务 Agent 都传入完整 System Prompt，同一份提示词会散落在多个服务中。模板修改以后，很难确认哪些调用已经升级，也无法从日志判断一次调用用了哪个版本。

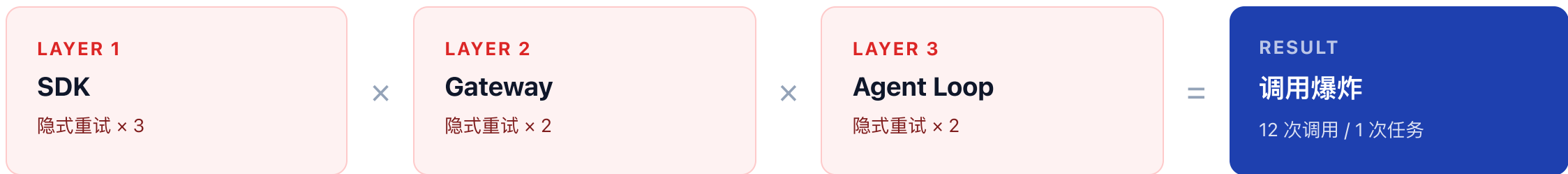
Gateway 集中管理受控模板

受控模板并不等于自动消除 Prompt 注入。变量仍可能来自不可信输入，是否允许变量进入 System Prompt、是否需要长度和内容限制，仍要按输入边界处理。



为什么重试必须由 Gateway 明确控制？

如果 **SDK**、**Gateway** 和 **Agent Loop** 都自动重试，一次任务可能产生远超预期的模型调用。



Gateway 关闭 SDK 隐式重试，明确控制重试次数和备用模型 fallback

为什么模型调用必须留下 Token、Cost 和 Latency?

模型调用除了状态和耗时，还要关注 Token。Token 同时影响成本、上下文窗口和输出长度。

T

Token

输入 + 输出 Token 数量，量化模型消耗

影响：成本 / 上下文窗口 / 输出长度

¥

Cost

按模型定价 × Token 换算费用

影响：预算控制 / 成本归因

ms

Latency

请求发出到响应完成的耗时

影响：用户体验 / SLA 监控

每次调用记录：模型 · Prompt 版本 · Token · Cost · Latency · 尝试次数 · 状态

怎样用 FastAPI 暴露统一入口?

BEFORE — Agent 直接调用

- ✗ 保存 DeepSeek API Key
- ✗ 拼接 Base URL
- ✗ 编写供应商重试逻辑

AFTER — 通过 Gateway 调用

- ✓ 提交平台模型名
- ✓ 选择模板
- ✓ 提交消息和业务 Schema

FastAPI 暴露三个入口:

POST
/chat
普通调用

POST
/stream
流式调用

GET
/trace
Trace 查询

基于 gateway.py 完成一次知识库 Agent 调用，验证以下场景

- 1 正常文本请求返回 LLMResponse
- 2 未知模型返回 unknown_model
- 3 模板版本不存在时返回 unknown_prompt_template
- 4 模板变量缺失时返回 missing_prompt_variable
- 5 合法 JSON 但不符合 Schema 时返回 schema_validation_failed
- 6 主模型连接失败时进行一次重试，再 fallback 到备用模型
- 7 流式请求逐块产生 content.delta
- 8 流开始后失败时产生 response.failed，不能静默换模型重写
- 9 成功和失败调用都能在 Trace 中找到
- 10 业务 Agent 的代码中不出现供应商 API Key 和 Base URL

验收时重点检查三条不变量：

不变量 1

未通过 Schema 校验的输出不能进入 Agent Loop

不变量 2

流已经输出首块以后，不能切换模型重新生成

不变量 3

无论使用主模型还是备用模型，上层只依赖同一份请求、响应和错误协议

如果这三条成立，Gateway 才真正成为 Harness 与模型服务之间的稳定边界

本节 LLM Gateway 解决「怎样让整个平台用同一种方式调用模型」

- 1 FastAPI 提供普通调用、Streaming 和 Trace 查询入口
- 2 Pydantic 在请求入口与响应出口执行协议校验
- 3 ModelConfig 和 Provider 隔离模型名、地址、密钥与兼容接口
- 4 Prompt 模板由 Gateway 按名称和版本渲染
- 5 Structured Output 根据模型能力选择 json_schema 或 json_object，返回后统一执行 Schema 校验
- 6 Gateway 关闭 SDK 隐式重试，明确控制重试次数和备用模型 fallback
- 7 每次调用记录模型、Prompt 版本、Token、Cost、Latency、尝试次数和状态

模型可以辅助编写代码，但不能替团队决定这些系统边界。能够把边界写成协议、校验、状态和失败路径，才是 LLM Gateway 对 Agent Harness 的真正价值。

LLM Gateway — Harness 与模型服务之间的稳定边界